



Fleet Operations: A Lambda-Architecture Data Pipeline for Ride-Hailing Utilisation and Profitability

*answering a live question and a financial question from one immutable
master dataset*

A mini-project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfilment of the requirements of

EC8203 — Applied Big Data Engineering

by

Ahamed N.F.F. - EG/2021/4390
Safeer S.M. - EG/2021/4771
Nafla M.N.P. - EG/2021/4687

Use Case 1: Ride-Hailing Fleet Operations
Academic Year 2026 | Mini-Project Report

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Objectives and Scope	1
2	Architecture and Technology Stack	2
2.1	Background: Lambda and Kappa	2
2.2	The Decision: Lambda	2
2.3	Why Kappa Was Rejected	2
2.4	What Lambda Costs, and Our Mitigation	3
2.5	Technology Stack	3
2.6	The Simulated Clock and Data Contract	3
3	Implementation	4
3.1	Ingestion	4
3.2	The Speed Layer	5
3.3	The Batch Layer and Orchestration	5
3.4	Serving	6
3.5	Observability	6
4	Results and Discussion	6
4.1	The Consistency Argument	6
4.2	Recomputation Demonstrated	7
4.3	Verification	7
4.4	Discussion	8
5	Conclusions and Recommendations	8
A	Alert Rule Inventory	9
B	API Surface	10
C	Statement of Individual Contributions	10

List of Figures

2.1	System architecture	3
4.1	Why the two layers disagree	7

List of Tables

1.1	The two sub-questions and their incompatible requirements.	1
2.1	Technology selection and the reason for each choice.	4
3.1	Injected fault rates, configured against measured ($\sim 2,700$ events).	5
4.1	Reconciliation outcomes over four simulated days.	7
A.1	The ten Prometheus alert rules. <i>Verified</i> means observed firing and delivered into <code>alert_notifications</code> .	9
B.1	The eleven documented endpoints, by the layer that answers them.	10
C.1	Principal areas of responsibility by team member.	11

Chapter 1

Introduction

A ride-hailing operator runs two businesses at once: an operations business conducted minute by minute (where are the cars, which are earning, which have been idle for an hour) and a financial business conducted a day at a time (what did each vehicle earn and cost, and is it still worth running). This project builds one platform that answers both without letting the fast answer contaminate the exact one.

1.1 Problem Statement

The brief specifies two ingestion paths. A **streaming source** emits GPS and telemetry per vehicle every few seconds (`trip_id`, `driver_id`, `vehicle_id`, position, speed, status, fare, timestamp). A **daily batch source** drops one file per simulated day of expense records from garages and fuel partners (`vehicle_id`, `fuel_cost`, `maintenance_cost`, `distance_covered`, `service_flag`). The telemetry is high-volume and tolerant of small losses; the expense file is owned by a third party, *authoritative about money*, and can be corrected and resubmitted days later, forcing published figures to be restated.

The business question — *what is utilisation and earnings by area right now, and which vehicles are becoming unprofitable once yesterday's costs are applied?* — is really two questions with conflicting requirements (Table 1.1).

Table 1.1: The two sub-questions and their incompatible requirements.

	Live utilisation	Daily profitability
Latency	seconds	next morning is acceptable
Accuracy	approximate is fine	exact; it is money
Recomputable?	no need	yes ; partners resubmit files
Consumer	dispatcher watching a map	finance, making retire/keep decisions

1.2 Objectives and Scope

The objectives were to choose between Lambda and Kappa on the evidence of this use case; to simulate both sources with injected faults so error handling is demonstrated rather than claimed; to perform real processing (cleaning, zone enrichment, windowed aggregation, stateful alerting and a cross-source join); to serve results through a store, API, dashboards and a daily report; and to instrument every stage, including the pipeline's *known imprecision*.

The whole system runs on one 16 GB laptop from one command, which forces three simplifications: simulated time is compressed 60-fold (Section 2.6), Kafka is a single broker at replication factor 1, and batch Spark runs in local mode. Authentication is out of scope.

Chapter 2

Architecture and Technology Stack

2.1 Background: Lambda and Kappa

Lambda (Marz and Warren) combines an *immutable master dataset*, a *batch layer* that recomputes exact views from it, and a *speed layer* that computes approximate views over recent data. Because batch views are a pure function of immutable input, errors are repaired by recomputing, and the speed layer’s error is temporary by construction. Kreps’s critique is that Lambda needs two implementations of every business rule, which can drift apart; his alternative, Kappa, keeps one stream processor and recomputes by *replaying the log*. That works only if everything needed to recompute a result is *in the log* — the assumption this use case breaks.

Three stream-processing results underpin the correctness argument: event time (when something happened) must be separated from processing time; a **watermark is a bet**, and events later than it are dropped, not deferred; and Spark’s exactly-once guarantee stops at the sink, so external writes from `foreachBatch` are at-least-once unless the sink is idempotent.

2.2 The Decision: Lambda

We implement Lambda: an archiver writes the immutable master dataset, a speed layer computes live views, and a batch layer recomputes exact views from the archive (Figure 2.1).

2.3 Why Kappa Was Rejected

The requirement is to *restate a past day’s financial figures* when a partner sends corrected costs. Under Kappa:

1. **Retention must cover any day we might restate** — months of Kafka retention. Ours is **7 days**, enough for speed-layer recovery only.
2. **Replay is not selective**. A Parquet lake partitioned by `sim_date` lets the batch job read one partition; a full simulated day is computed in **9.4 seconds**.

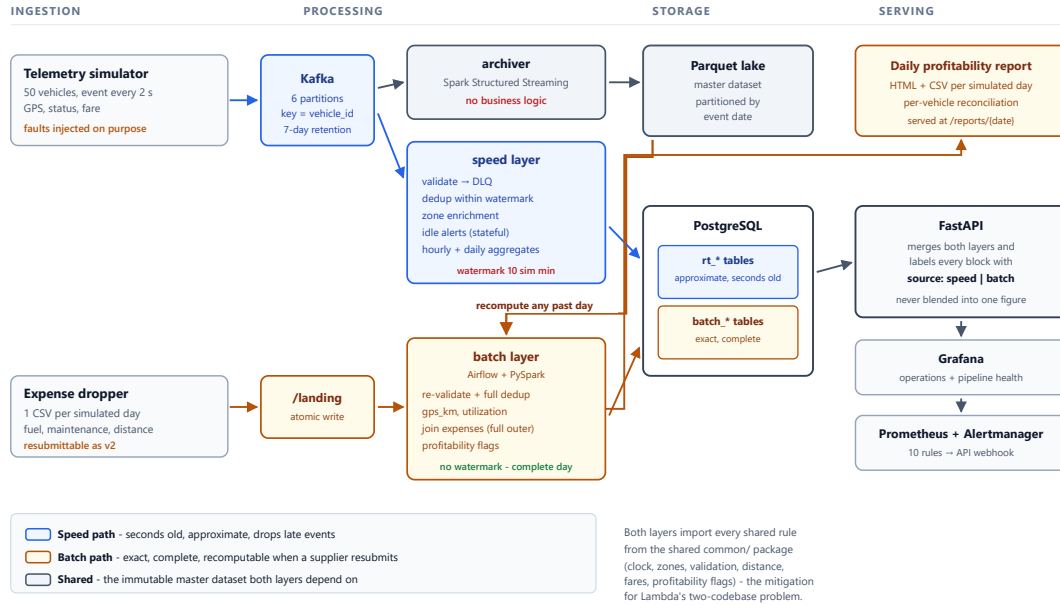


Figure 2.1: System architecture. Blue: speed layer (seconds old, approximate). Amber: batch layer (exact, recomputable). Both read the same master dataset and import shared rules from `common/`.

3. **The correction is not in the stream at all.** It arrives as a CSV file on a separate channel, so replaying the log does not touch what actually forces a restatement. Publishing the file to a topic only moves the problem back to point 1.

A batch-only design was rejected because it cannot say which vehicles are idle *now*.

2.4 What Lambda Costs, and Our Mitigation

We accept Kreps's criticism and answer it structurally. A shared `common/` package holds every rule both layers need — simulated clock, zone mapping, validation, haversine distance, fare maths and the seven profitability flags — and is imported by the simulators, both processing layers and the API. Only two rules are duplicated, to avoid a per-row Python UDF: validation (`validate_event()` vs. `spark_validation_expr()`) and haversine distance. Each pair is held in step by a test that feeds identical inputs to both.

2.5 Technology Stack

Every image and package version is pinned (Table 2.1).

2.6 The Simulated Clock and Data Contract

One real second is one simulated minute (`COMPRESSION=60`), so a simulated day takes 24 real minutes and a daily reconciliation can be shown in a lab session. The clock is derived from a persisted start instant, so every service agrees on `sim_time` and restarts do not rewind it.

We added three telemetry fields: `event_id` (UUID, for deduplication), `schema_`

Table 2.1: Technology selection and the reason for each choice.

Layer	Choice	Why
Ingestion	Kafka 3.8 (KRaft)	Per-partition ordering is a correctness requirement; KRaft drops ZooKeeper.
Speed layer	Spark Structured Streaming 3.5.3	Watermarks, windows and <code>applyInPandasWithState</code> ; one engine for both layers makes common/ shareable (preferred over Flink for that reason).
Master dataset	Parquet on a volume	Columnar, date-partitioned; paths built from <code>LAKE_ROOT</code> , so moving to S3 is one variable.
Batch	Airflow 2.10 + PySpark	Retries, dependency ordering, per-task visibility; local mode suits $\sim 12k$ events/day.
Serving	PostgreSQL 16, FastAPI	Upserts on natural keys give idempotent sinks; Pydantic models make the source label part of the schema.
Observability	Prometheus, Alertmanager, Grafana	Pull-based metrics, routed alerts, dashboards.

`version`, and `event_type` (`ping/trip_start/trip_end`), which makes “trips completed” a simple count instead of a chained stateful operator. **fare is cumulative and final only on `trip_end`**, so revenue is `SUM(fare) WHERE event_type='trip_end'` in both layers.

Chapter 3

Implementation

3.1 Ingestion

3.1.1 Partitioning and Fault Injection

`fleet.telemetry` has 6 partitions **keyed by `vehicle_id`**. Kafka orders events only within a partition, and the per-vehicle idle state machine assumes it sees each vehicle’s events in order, so keying is a correctness decision, not a throughput one. Rejected

events go to `fleet.telemetry.dlq` with a reason code and the original payload, so they can be replayed.

The simulator corrupts its own output (Table 3.1). Validation *rejects rather than repairs*: a coerced field would produce a confidently wrong money figure. A second simulator drops one versioned expense CSV per simulated day; a corrected `v2` for a reconciled day triggers recomputation (Section 4.2).

Table 3.1: Injected fault rates, configured against measured ($\sim 2,700$ events).

Fault	Config.	Measured	What it proves
Malformed JSON	0.5%	0.40%	Archiver keeps raw bytes; speed layer dead-letters instead of crashing
Schema-invalid	0.5%	0.77%	Shared validation rejects identically in both layers
Duplicate <code>event_id</code>	1.0%	1.03%	Deduplication works
Late by 1–20 sim min	2.0%	2.09%	The consistency argument (Section 4.1)

3.2 The Speed Layer

Archiver. A separate Spark application with its own checkpoint and *no business logic*, so a bug in the speed layer cannot corrupt the master dataset. Unparseable records are kept under `sim_date=unknown`.

Stateful idle alerts. “Idle for 45 simulated minutes?” needs per-vehicle memory across micro-batches and must fire even when no event arrives, so it uses `applyInPandasWithState`. The timer starts on the *transition* into idle; events are sorted by event time before folding; a stale out-of-order event does not rewind state; and an `EventTimeTimeout` is used so the rule does not depend on clock compression. Because event-time timeouts need the watermark to advance, a fleet-wide silence is covered separately by the `TelemetryNotProduced` alert.

Idempotent sinks. `foreachBatch` is at-least-once, so every sink upserts on a natural key (`rt_vehicle_state` on `vehicle_id`, `rt_zone_hourly` on (`zone_id`, `window_start`), `rt_vehicle_daily` on (`vehicle_id`, `sim_date`), `idle_alerts` on (`vehicle_id`, `opened_at`)). A replayed batch rewrites the same values, and `update` output mode shows the current hour live.

3.3 The Batch Layer and Orchestration

`compute_vehicle_day` re-reads the raw archive and re-applies validation and deduplication itself, with **no watermark**; it never reads speed-layer output, so the drift measurement compares two independent computations. It produces per vehicle-day trips, revenue, GPS distance, online/on-trip/idle minutes and utilisation. Telemetry and expenses are joined with a **full outer join** so that vehicles with no cost record (`missing_costs`) and cost records with no telemetry (`missing_telemetry`) are flagged rather than silently dropped.

Airflow runs in real time, so the nine-task DAG polls every 2 real minutes and its first task picks the pending simulated day. `load_batch_views` deletes and re-inserts a date’s rows in one transaction, making recomputation idempotent and atomic.

3.4 Serving

Both layers write to one PostgreSQL instance. FastAPI exposes eleven endpoints (Appendix B), including the daily report at `GET /reports/{sim_date}`, and two Grafana dashboards read the same tables. `GET /vehicles/{id}` returns live and today blocks labelled "source": "speed" and a history block labelled "source": "batch" — never blended, so a dashboard estimate cannot end up in a financial statement.

3.5 Observability

Metrics answer four questions: *is data arriving* (`fleet_events_produced_total`), *is it processed* (per-query `fleet_stream_last_progress_timestamp`), *is it correct* (`fleet_dlq_events_total`, `fleet_late_rows_dropped_total`, `fleet_speed_batch_drift_ratio`), and *is the business healthy* (`fleet_open_idle_alerts`). Kafka lag is read from Spark's query progress, because Structured Streaming does not commit consumer-group offsets. Batch outcomes are exported by the API from PostgreSQL instead of a Pushgateway, since Airflow tasks exit before Prometheus scrapes.

Every log line is a JSON object with `service`, `stage`, `event` and `sim_time`. Instead of distributed tracing we propagate correlation IDs: `event_id`, `vehicle_id`, `run_id` and `sim_date` link a DLQ entry or a reconciled figure back to its source. Ten alert rules are defined (Appendix A).

Chapter 4

Results and Discussion

All figures were read from the running stack, started from a fresh clone with `make up`; the DAG reconciled four simulated days unattended.

4.1 The Consistency Argument

The speed layer uses a 10-simulated-minute watermark while the simulator injects lateness of up to 20 minutes, so the layers are *built* to disagree (Figure 4.1); a regression test stops anyone widening the watermark to hide this.

On 2024-01-02 the speed layer counted 316 trips against the batch layer's 317: one late `trip_end` dropped by the watermark and recovered by the batch layer. Drift is usually zero because, per day, the watermark drops about 15 rows and only 3.4% of events are `trip_end`, so the expected number of lost revenue events is $15 \times 0.034 \approx 0.5$. `fleet_late_rows_dropped_total` therefore shows the *mechanism* (reliably non-zero)

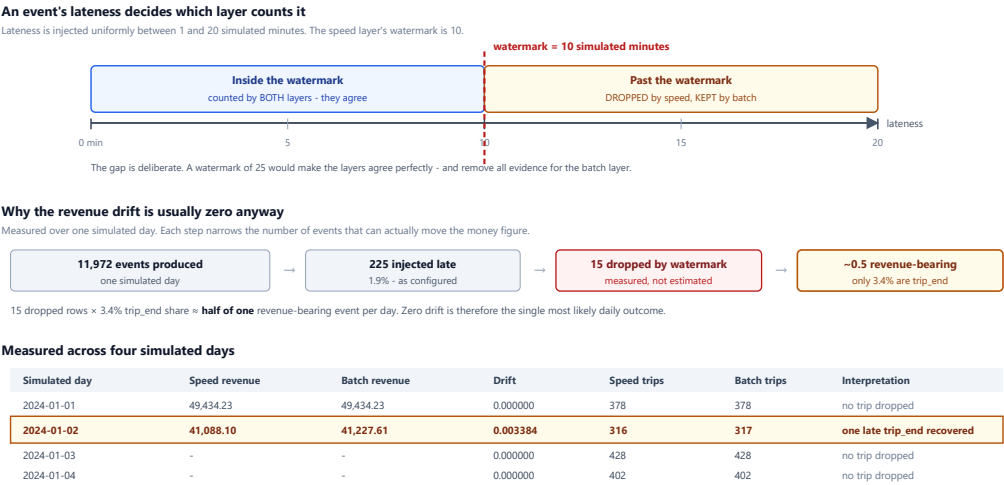


Figure 4.1: Why the two layers disagree: an event's lateness decides which layer counts it.

Table 4.1: Reconciliation outcomes over four simulated days.

Simulated day	Fleet margin	Unprof.	Becoming unprof.	Dist. mismatch	Revenue drift
2024-01-01 (partial)	42.8%	6	0	2	0.000000
2024-01-02	38.3%	4	0	21	0.003384
2024-01-03	45.8%	3	6	6	0.000000
2024-01-04	47.7%	4	7	4	0.000000

and `fleet_speed_batch_drift_ratio` its *financial weight* (usually zero). The case for batch-computed money is not that the speed layer is badly wrong, but that its error is unbounded in principle, and no watermark can absorb a correction arriving weeks later.

4.2 Recomputation Demonstrated

Dropping a `v2` expense file for a reconciled day made it pending again; the recompute produced **the same 50 rows with new values** (net profit 19,770.78 $\rightarrow$ 21,177.99). A forced rerun of 2024-01-03 returned **bit-identical figures** (net 24,902.95). The restatement came from one Parquet partition, with no log replay.

4.3 Verification

- 202 unit, PySpark and API tests pass; the smoke test passes 28/28 checks; all 13 services start healthy from a fresh clone.
- 5 of 10 alert rules were observed firing *and* delivered; `BatchRunFailed` is deliberately inhibited when `ExpenseFileLate` explains the same failure.
- 23 of 23 Grafana panel queries return live data.
- Batch task times: `compute_vehicle_day` 9.44 s, `load_batch_views` 0.99 s, all others under 0.5 s.

4.4 Discussion

Defects found by operating the system. Six defects surfaced only at runtime. The most serious: a Spark worker restart killed the archiver’s executor while the query stayed “active”, so the **healthcheck stayed green** while nothing was archived for 26 minutes (fixed by a progress watchdog). Others: an odometer checkpoint every 30 real seconds (30 simulated minutes) that caused false distance mismatches; uncalibrated economics flagging 35 of 50 vehicles unprofitable; batch gauges exported by every service, firing a permanent false alert; an expense SLA applied to corrected `v2` files; and two Windows-only start-up bugs.

Limitations. Four days is one run of one configuration, and day 1 is partial. `becoming_unprofitable` needs three reconciled days to fire. Batch Spark runs in local mode and Kafka has a single broker at RF 1. Because simulated time follows the wall clock, downtime appears in the data as low demand.

At production scale. Kafka would move to three brokers at RF 3 with a schema registry (keeping the `vehicle_id` key); the lake would use object storage with a table format such as Iceberg or Delta and compaction; processing would run on Kubernetes with the archiver separated from the speed layer; and observability would add log aggregation, tracing, and SLOs with ratio-based thresholds.

Chapter 5

Conclusions and Recommendations

The brief’s question is two questions with incompatible latency, accuracy and re-computability requirements. Lambda is justified here by one property Kappa cannot supply cheaply: **restating a past day’s figures when a correction arrives on a different channel, days later**. A corrected file produced the same 50 rows with updated values, and a forced replay was bit-identical. We measured the speed layer’s approximation rather than asserting it, explained why it is usually zero, and contained Lambda’s two-codebase weakness with the shared `common/` package and parity tests.

Recommendations

1. Run the archiver and speed layer on separate workers.
2. Add a schema registry and an automated DLQ replay path.
3. Adopt a lake table format (Iceberg or Delta Lake) for atomic partition writes.
4. Annotate outage windows and use ratio-based alert thresholds.

Appendix A

Alert Rule Inventory

Table A.1: The ten Prometheus alert rules. *Verified* means observed firing **and** delivered into `alert_notifications`.

Alert	Stage	Sev.	Condition	Verified
TelemetryNotProduced	ingestion	critical	No events produced for over a minute	yes
DLQRateHigh	ingestion	warning	More than 2% of events dead-lettered	yes
StreamStalled	processing	critical	A named query has stopped progressing	–
StreamLagHigh	processing	warning	Query more than 5,000 off-sets behind	–
ExpenseFileLate	orchestration	warning	Expense file missed its first-delivery SLA	yes
BatchRunFailed	orchestration	critical	Last reconciliation run inhibited failed	–
BatchNotRunRecently	orchestration	warning	No successful reconciliation in 45 minutes	yes
SpeedBatchDriftHigh	processing	warning	Speed and batch revenue differ by over 5%	–
ApiDown	serving	critical	The serving API is not reachable	yes
IdleVehiclesHigh	serving	warning	More than 8 vehicles idle past threshold	–

Appendix B

API Surface

Table B.1: The eleven documented endpoints, by the layer that answers them.

Endpoint	Layer	Answers
GET /health	ops	Liveness, <code>sim_time</code> , compression
GET /fleet/live	speed	Active vehicles, idle ratio, trips/hour, earnings
GET /zones/live	speed	Current utilisation and earnings by zone
GET /zones/{id}/hourly	speed	One zone's hourly history
GET /alerts/idle	speed	Open and closed idle alerts
GET /vehicles/unprofitable	batch	Vehicles flagged (becoming) unprofitable
GET /vehicles/{id}	both	Labelled speed and batch blocks
GET /reports/{sim_date}	batch	Daily reconciliation report
GET /pipeline/status	ops	Reconciled dates, last run, drift
POST /alerts/webhook	ops	Alertmanager delivery target
GET /metrics	ops	Prometheus exposition

Appendix C

Statement of Individual Contributions

Work was divided by **pipeline stage**, so each member owned one vertical slice: its code, schema objects, tests and report sections. Design decisions were taken jointly and code was reviewed across the team.

Worked on jointly: `docs/SPEC.md`, `docker-compose.yml`, the `Makefile`, the smoke test, the architecture and consistency analysis, and every end-to-end acceptance run. A fuller breakdown is in `docs/contributions.md`.

Table C.1: Principal areas of responsibility by team member.

Member	Reg. No.	Principal areas
Ahamed N.F.F.	EG/2021/4390	Ingestion and data contract: <code>simulators/</code> (telemetry producer, fault injection, expense dropper), Kafka topics and partitioning, and the <code>simclock</code> , <code>schemas</code> , <code>validation</code> , <code>geo</code> and <code>earnings</code> modules of <code>common/</code>
Safeer S.M.	EG/2021/4771	Stream processing and serving: <code>streaming/</code> (archiver, speed layer, idle state machine, sinks, watchdog), <code>api/</code> , the <code>rt_*</code> tables and the <code>db</code> helpers in <code>common/</code>
Nafla M.N.P.	EG/2021/4687	Batch, orchestration and observability: <code>batch/</code> , the <code>fleet_daily_reconciliation</code> DAG, <code>observability/</code> (Prometheus rules, Alertmanager, Grafana), and the <code>profitability</code> , <code>metrics</code> and <code>logging</code> modules of <code>common/</code>